

Mu Runtime Reference

version 0.2.22

type keywords and aliases

<i>supertype</i>	<i>T</i>	
<i>bool</i>	<i>()</i> , <i>nil</i>	are false, else true
<i>condition</i>	<i>keyword</i>	see exceptions
<i>list</i>	<i>:cons</i> or <i>()</i> , <i>nil</i>	
<i>ns</i>	<i>#s(:ns #(t fixnum symbol))</i>	
<i>ns-designator</i>	<i>ns</i>	
<i>:null</i>	<i>()</i> , <i>nil</i>	
<i>:char</i>	<i>char</i>	8 bit ASCII
<i>:cons</i>	<i>cons</i> , <i>list</i>	list, cons, dotted pair
<i>:fixnum</i>	<i>fixnum</i> , <i>fix</i>	56 bit signed integer
<i>:float</i>	<i>float</i> , <i>fl</i>	32 bit IEEE float
<i>:func</i>	<i>function</i> , <i>fn</i>	function
<i>:keyword</i>	<i>keyword</i> , <i>key</i>	symbol
<i>:stream</i>	<i>stream</i>	file or string type
<i>:struct</i>	<i>struct</i>	see structs
<i>:symbol</i>	<i>symbol</i> , <i>sym</i>	LISP-1 symbol
<i>:vector</i>	<i>vector</i> , <i>string</i> , <i>str</i>	typed vector
	<i>:bit</i> :char :t	
	<i>:</i>	byte :fixnum :float

core

apply <i>fn list</i>	<i>T</i>	apply <i>fn</i> to <i>list</i>
compile <i>form</i>	<i>T</i>	<i>mu</i> form compiler
eq <i>T T'</i>	<i>bool</i>	<i>T</i> and <i>T'</i> identical?
eval <i>form</i>	<i>T</i>	evaluate <i>form</i>
type-of <i>T</i>	<i>key</i>	type keyword
view <i>T</i>	<i>vector</i>	vector of object
fix <i>fn T</i>	<i>T</i>	fixpoint of <i>fn</i>
gc	<i>bool</i>	garbage collection
repr <i>T</i>	<i>vector</i>	tag representation
unrepr <i>vector</i>	<i>T</i>	tag representation

tag *vector* is an 8 element :byte vector of little-endian argument tag bits.

special forms

<i>:lambda list . list'</i>	<i>function</i>	anonymous <i>fn</i>
<i>:alambda list . list'</i>	<i>function</i>	anonymous <i>fn</i>
<i>:quote T</i>	<i>list</i>	quoted form
<i>:if T T' T''</i>	<i>T</i>	conditional

frames

frame binding: (*fn* . #(t ...))

%frame-stack	<i>list</i>	active frames
%frame-pop <i>fn</i>	<i>frame</i>	pop function's top frame binding
%frame-push <i>frame</i>	<i>cons</i>	push frame
%frame-ref <i>fn fix</i>	<i>T</i>	function, offset

symbols

boundp <i>sym</i>	<i>bool</i>	is <i>symbol</i> bound?
make-symbol <i>string</i>	<i>sym</i>	uninterned <i>symbol</i>
symbol-namespae <i>sym</i>	<i>ns-designator</i>	
	<i>namespace designator</i>	
symbol-name <i>symbol</i>	<i>string</i>	name binding
symbol-value <i>symbol</i>	<i>T</i>	value binding

fixnums

add <i>fix fix'</i>	<i>fixnum</i>	sum
ash <i>fix fix'</i>	<i>fixnum</i>	arithmetic shift
div <i>fix fix'</i>	<i>fixnum</i>	quotient
less-than <i>fix fix'</i>	<i>bool</i>	<i>fix</i> < <i>fix'</i> ?
logand <i>fix fix'</i>	<i>fixnum</i>	bitwise and
lognot <i>fix</i>	<i>fixnum</i>	bitwise complement
logor <i>fix fix'</i>	<i>fixnum</i>	bitwise or
mul <i>fix fix'</i>	<i>fixnum</i>	product
sub <i>fix fix'</i>	<i>fixnum</i>	difference

floats

fadd <i>fl fl'</i>	<i>float</i>	sum
fdiv <i>fl fl'</i>	<i>float</i>	quotient
fless-than <i>fl fl'</i>	<i>bool</i>	<i>fl</i> < <i>fl'</i> ?
fmul <i>fl fl'</i>	<i>float</i>	product
fsub <i>fl fl'</i>	<i>float</i>	difference

conses/lists

append <i>list</i>	<i>list</i>	append lists
car <i>list</i>	<i>T</i>	head of <i>list</i>
cdr <i>list</i>	<i>T</i>	tail of <i>list</i>
cons <i>T T'</i>	<i>cons</i>	(<i>T</i> . <i>T'</i>)
length <i>list</i>	<i>fixnum</i>	length of <i>list</i>
nth <i>fix list</i>	<i>T</i>	<i>nth</i> car of <i>list</i>
nthcdr <i>fix list</i>	<i>T</i>	<i>nth</i> cdr of <i>list</i>

vectors

make-vector <i>key list</i>	<i>vector</i>	specialized <i>vector</i> from <i>list</i>
vector-length <i>vector</i>	<i>fixnum</i>	length of <i>vector</i>
vector-type <i>vector</i>	<i>key</i>	type of <i>vector</i>
svref <i>vector fix</i>	<i>T</i>	<i>nth</i> element

namespaces

mu (static), **keyword**, **ns**: *#s(:ns #(t str))*

make-namespace <i>str</i>	<i>ns</i>	make namespace
namespace-name <i>ns</i>	<i>string</i>	namespace name
intern <i>ns str value</i>	<i>symbol</i>	intern <i>symbol</i> in non-static namespace
find-namespace <i>str</i>	<i>ns</i>	map <i>string</i> to namespace
find <i>ns string</i>	<i>symbol</i>	map <i>string</i> to <i>symbol</i>

structs

make-struct <i>key list</i>	<i>struct</i>	type <i>key</i> from <i>list</i>
struct-type <i>struct</i>	<i>key</i>	<i>struct</i> type <i>key</i>
struct-vec <i>struct</i>	<i>vector</i>	of <i>struct</i> members

streams

standard-input	<i>stream</i>	std input <i>stream</i>
standard-output	<i>stream</i>	std out <i>stream</i>
error-output	<i>stream</i>	std error <i>stream</i>
open <i>type dir str bool</i>	<i>stream</i>	open <i>stream</i> , raise error if <i>bool</i>
	<i>type</i> :file :string	
	<i>dir</i> :input :output :bidir	
close <i>stream</i>	<i>bool</i>	close <i>stream</i>
openp <i>stream</i>	<i>bool</i>	is <i>stream</i> open?
flush <i>stream</i>	<i>bool</i>	flush <i>stream</i>
get-string <i>stream</i>	<i>string</i>	from <i>string</i> <i>stream</i>
read-byte <i>stream bool T</i>	<i>byte</i>	read <i>byte</i> from <i>stream</i> , error on eof, <i>T</i> : eof-value
read-char <i>stream bool T</i>	<i>char</i>	read <i>char</i> from <i>stream</i> , error on eof, <i>T</i> : eof-value
unread-char <i>char stream char</i>		push <i>char</i> onto <i>stream</i>
write-byte <i>byte stream</i>	<i>byte</i>	write <i>byte</i>
write-char <i>char stream</i>	<i>char</i>	write <i>char</i>
read <i>stream bool T</i>	<i>T</i>	read <i>stream</i>
write <i>T bool stream</i>	<i>T</i>	write with escape

exceptions

with-exception *fn fn'* *T* catch exception

fn - (:lambda (*obj cond src*) . *body*)

fn' - (:lambda () . *body*)

raise *T T' keyword* raise exception on *T* from source designator *T'* with *keyword* condition

:arity :div0 :eof :error :except

:future :ns :open :over :quasi

:range :read :exit :signal :stream

:syntax :syscall :type :unbound :under

:write :storage :user

Features

[features]

default = ["core", "env", "system"]

feature/core	core-info	<i>list</i>	core state
	process-fds	<i>list</i>	file descriptors
	process-mem-virt	<i>fixnum</i>	vmem
	process-mem-res	<i>fixnum</i>	reserve
	process-time	<i>fixnum</i>	microseconds
feature/env	time-units-per-sec	<i>fixnum</i>	
	sleep	<i>fixnum</i>	
		<i>()</i>	sleep for usecs
	env-info	<i>list</i>	env info
	heap-info	<i>list</i>	list of heap info
feature/system	heap-room	<i>key vector</i>	allocations
		<i>#(:t size total free ...)</i>	
	heap-size	<i>key fixnum</i>	type size
	cache-room	<i>vector</i>	allocations
		<i>#(:t size total ...)</i>	
feature/socket	load	<i>string bool</i>	load mu source
	symbols	<i>ns list</i>	ns symbol list
	uname	<i>:t</i>	system info
	shell	<i>string list fixnum</i>	shell command
	exit	<i>fixnum</i>	doesn't return
feature/ffi	sysinfo	<i>:t</i>	not on macOS
	accept		
	bind		
	connect...		
	info		

environment config

JSON config format:

```
{
  "pages": N,
  "gc-mode": "none" | "auto",
}
```

Mu library API

[dependencies]

```
mu = {
  git = "https://github.com/Software-Knife-and-Tool/mu.git",
  branch = "main"
}
```

```
use mu:: { Mu, Env, Config };
use mu:: { Result, Tag, Exception, Condition };
```

```
impl Mu {
  fn compile(_: &Env, _: Tag) → Result<Tag>
  fn config(_: Option<String>) → Config
  fn env(_: &Config) → Env
  fn eq(_: Tag, _: Tag) → bool;
  fn err_out() → Tag
  fn eval_str(_: &Env, _: &str) → Result<Tag>
  fn eval(_: &Env, _: Tag) → Result<Tag>
  fn exception_string(_: &Env, _: Exception) → String
  fn load(_: &Env, _: &str) → Result<bool>
  fn env(_: &Config) → Env
  fn nil() → Tag
  fn read_str(_: &Env, _: &str) → Result<Tag>
  fn read(_: &Env, _: Tag, _: bool, _: Tag) → Result<Tag>
  fn std_in() → Tag
  fn std_out() → Tag
  fn version() → &str
  fn write_str(_: &Env, _: &str, _: Tag) → Result<()>
  fn write_to_string(_: &Env, _: Tag, _: bool) → String
  fn write(_: &Env, _: Tag, _: bool, _: Tag) → Result<()>
}
```

Reader Syntax

```
; comment to end of line
#|...|# block comment
'form quoted form
`form backquoted form
`(...) backquoted list (proper lists)
,form eval backquoted form
,@form eval-splice backquoted form
(...) constant list
() empty list, prints as :nil
(...) .) dotted list
"..." string, char vector
| single escape in strings
ns:name qualified symbol, where ns and name are symbol constituents
name lexical symbol
** bit vector
#x hexadecimal fixnum
#. read-time eval
#\ char
#(:type ...) vector
#s(:type ...) struct
#:... uninterned symbol
"`,; terminating macro char
# non-terminating macro char

!$%*+-. symbol constituent
<=>=?@[|
: ^_{ } ~ /
A..Za..z0..9
```

```
0x09 #\tab character designators
0x0a #\linefeed
0x0c #\page
0x0d #\return
0x20 #\space
```